



RV College of
Engineering®

Go, Change the World

Academic Year 2024-25 (ODD Semester)

USN

1	R	V	2	2	A	5	0	0	7
---	---	---	---	---	---	---	---	---	---

Department of Artificial Intelligence and Machine Learning

ARTIFICIAL NEURAL NETWORKS AND DEEP LEARNING

Course Code : AI253IA

Date : 7/1/2025

Semester : V Sem

Time : 9:30am to 11:30am

Max Marks : 10 (Q) + 50 (CIE)

Duration : 30 + 90=120 min

QUIZ 2

Note: Answer all the Questions

Q. No		Questions	M	BT	CO																
1	a)	What is the purpose of the stride parameter in a convolutional layer?	2	2	1																
	b)	Given the values for input gate ($i_t=0.8$), forget gate ($f_t=0.6$), output gate ($o_t=0.7$), previous cell state ($c_{t-1}=0.5$), and candidate cell state ($ct=4$), calculate the updated cell state ct using the LSTM cell state update equation	2	1	1																
	c)	Given $x_t=1.5, h_{t-1}=0.8, W_{xh}=0.4, W_{hh}=0.6$, and $b_h=0.2$ calculate the hidden state h_t at time step t using the RNN update equation	2	3	1																
	d)	List any two methods of data augmentation techniques commonly used for image data.	2	2	1																
	e)	Give the max pooling considering 2×2 window size <table><tr><td>3</td><td>1</td><td>1</td><td>3</td></tr><tr><td>2</td><td>5</td><td>0</td><td>2</td></tr><tr><td>1</td><td>4</td><td>2</td><td>1</td></tr><tr><td>4</td><td>7</td><td>2</td><td>4</td></tr></table>	3	1	1	3	2	5	0	2	1	4	2	1	4	7	2	4	2	2	1
3	1	1	3																		
2	5	0	2																		
1	4	2	1																		
4	7	2	4																		

CIE 2

Note: Answer all the Questions

Q. No	Questions	M	BT	CO
2 a)	How is backpropagation implemented through matrix multiplication in Convolutional Neural Networks (CNNs)?	5	3	1
b)	How do pre trained convolutional neural network handles object localization? With neat diagram elaborate the steps in detail	5	2	1
3 a)	Consider a simple Recurrent Neural Network (RNN) with the following setup: The RNN has one hidden layer with a hidden state vector h_t which at each time step t is updated by the equation: $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ Given the loss function	5	3	3

		$L = \sum_{t=1}^T \frac{1}{2} (y_t - \hat{y}_t)^2$																																																
		Compute the gradient of the loss with respect to W_{hy} and W_{hh} (the output weight matrix) using BPTT.																																																
	b)	Provide a comprehensive technical overview of a Convolutional Neural Network (CNN) architecture. Please include a brief explanation of each layer, such as the input layer, convolutional layers, activation functions, pooling layers, fully connected layers, and the output layer.	5	2	1																																													
4	a)	In a smart grid energy management system, an Echo State Network (ESN) is used to forecast energy demand based on real-time data from IoT sensors. The system receives input features such as current energy consumption, weather conditions, and time of day. The ESN is trained to predict energy demand for the next hour. a) Describe the architecture of the ESN in this scenario, highlighting the roles of the input, reservoir, and output layers.	5	3	3																																													
	b)	Consider the statement, "Captain Rohit fail in BGT Test Series." Discuss briefly how a Long Short-Term Memory (LSTM) network can be effectively utilized for grammatical error detection and correction in such sentences. Provide the architecture diagram of an LSTM cell and provide mathematical equations for the cell state	5	3	1																																													
5	a)	Provide a detailed explanation of the mathematical formulations for Bidirectional and Multilayer Recurrent Neural Networks (RNNs), including the state transition equations for both forward and backward passes in BRNNs, and the effects of stacking multiple recurrent layers in Multilayer RNNs."	5	3	2																																													
	b)	Consider a machine translation task where the goal is to translate a sentence from English to Kannada. Identify the most appropriate type of RNN for this task, and explain its architecture and justify your choice	5	4	3																																													
6		Considering the given matrix i) Perform the convolution operation with the given filter and stride = 1 and zero padding and discuss the output in detail. <table><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr></table> Filter 3×3 <table><tr><td>3</td><td>0</td><td>1</td><td>2</td><td>7</td><td>4</td></tr><tr><td>1</td><td>5</td><td>8</td><td>9</td><td>3</td><td>1</td></tr><tr><td>2</td><td>7</td><td>2</td><td>5</td><td>1</td><td>3</td></tr><tr><td>0</td><td>1</td><td>3</td><td>1</td><td>7</td><td>8</td></tr><tr><td>4</td><td>2</td><td>1</td><td>6</td><td>2</td><td>8</td></tr><tr><td>2</td><td>4</td><td>5</td><td>2</td><td>3</td><td>9</td></tr></table> Input 6×6	1	0	-1	1	0	-1	1	0	-1	3	0	1	2	7	4	1	5	8	9	3	1	2	7	2	5	1	3	0	1	3	1	7	8	4	2	1	6	2	8	2	4	5	2	3	9	1 0	3	2
1	0	-1																																																
1	0	-1																																																
1	0	-1																																																
3	0	1	2	7	4																																													
1	5	8	9	3	1																																													
2	7	2	5	1	3																																													
0	1	3	1	7	8																																													
4	2	1	6	2	8																																													
2	4	5	2	3	9																																													

M-Marks, BT-Blooms Taxonomy Levels, CO-Course Outcomes

Marks Distribution	Particulars	CO1	CO2	CO3	CO4	L1	L2	L3	L4	L5	L6
	Max Marks CIE & Quiz	30	5	15	-	4	12	20	14	10	-

Course Outcomes

CO1:	Describe basic concepts of neural networks, its applications and various learning models
CO2:	Analyze different network architectures, learning tasks, CNN, and deep learning models
CO3:	Investigate and apply neural networks model and learning techniques to solve problems related to society and industry.
CO4:	Demonstrate a prototype application developed using any NN tools and APIs.
CO5:	Appraise the knowledge of neural networks and deep learning as an individual/as an team member.

Previous Test - Quiz Solutions

Date: 7th January 2025

1. What is the purpose of the stride parameter in a convolutional layer?

The **stride** parameter controls the step size of the kernel (filter) as it slides across the input image/matrix.

- **Spatial Downsampling:** Increasing the stride (e.g., Stride = 2) reduces the spatial dimensions (height and width) of the output feature map, typically by a factor of the stride. This reduces computational cost and the number of parameters in subsequent layers.
- **Receptive Field:** Larger strides effectively increase the receptive field of neurons in deeper layers more quickly, allowing them to cover larger areas of the input image.

2. Given the values for input gate ($i_t = 0.8$), forget gate ($f_t = 0.6$), output gate ($o_t = 0.7$), previous cell state ($C_{t-1} = 0.5$), and candidate cell state ($\tilde{C}_t = 4$), calculate the updated cell state C_t using the LSTM cell state update equation.

Equation: The updated cell state C_t is a combination of the previous memory (scaled by the forget gate) and the new candidate memory (scaled by the input gate).

$$C_t = (f_t \times C_{t-1}) + (i_t \times \tilde{C}_t)$$

Calculation:

- $f_t \times C_{t-1} = 0.6 \times 0.5 = 0.3$
- $i_t \times \tilde{C}_t = 0.8 \times 4 = 3.2$
- $C_t = 0.3 + 3.2$

Answer:

$$C_t = 3.5$$

3. Given $x_t = 1.5$, $h_{t-1} = 0.8$, $W_{xh} = 0.4$, $W_{hh} = 0.6$, $b_h = 0.2$, calculate the hidden state h_t at time step t using the RNN update equation.

Equation: The standard RNN hidden state update equation is:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

Step 1: Calculate the linear combination (activation argument)

$$Z = (0.4 \times 1.5) + (0.6 \times 0.8) + 0.2$$

$$Z = 0.6 + 0.48 + 0.2$$

$$Z = 1.28$$

Step 2: Apply the activation function (tanh)

$$h_t = \tanh(1.28)$$

Using a calculator, $\tanh(1.28) \approx 0.856$

Answer:

$$h_t \approx 0.856$$

4. List any 2 methods of data augmentation techniques commonly used for image data.

1. **Horizontal/Vertical Flipping:** Mirroring the image across the horizontal or vertical axis. This is useful for datasets where orientation doesn't change the class (e.g., a cat is still a cat if looked at in a mirror).
2. **Rotation:** Rotating the image by a random angle (e.g., between -30° and $+30^\circ$). This helps the model become invariant to the orientation of the object.

(Other common methods include: Cropping, Zooming, Shearing, and Color Jittering).

5. Max pooling of 2x2 matrix:

Input Matrix:

$$\begin{bmatrix} 3 & 1 & 1 & 3 \\ 2 & 5 & 0 & 2 \\ 1 & 4 & 2 & 1 \\ 4 & 7 & 2 & 4 \end{bmatrix}$$

Assumption: Standard Max Pooling with a 2×2 filter and Stride = 2.

Calculation: We divide the matrix into four 2×2 regions and select the maximum value from each.

1. **Top-Left Block:** $\max(3, 1, 2, 5) \rightarrow 5$
2. **Top-Right Block:** $\max(1, 3, 0, 2) \rightarrow 3$
3. **Bottom-Left Block:** $\max(1, 4, 4, 7) \rightarrow 7$
4. **Bottom-Right Block:** $\max(2, 1, 2, 4) \rightarrow 4$

Output Matrix:

$$\begin{bmatrix} 5 & 3 \\ 7 & 4 \end{bmatrix}$$

6. How is Backpropagation implemented through matrix multiplication in Convolutional Neural Networks (CNN)?

In CNNs, backpropagation computes the gradient of the loss function with respect to the weights (filters) and the inputs. To make this computationally efficient, convolution operations are often transformed into matrix multiplications (using techniques like `im2col`).

1. Forward Pass (Matrix View): Input volumes are "unrolled" into a large matrix where each column corresponds to a patch of the image that the filter will act upon. The filters are flattened into rows.

$$Y = W_{row} \cdot X_{col}$$

2. Backward Pass (Gradients): Assume we have the gradient of the loss from the next layer, denoted as δ^{l+1} (or $\frac{\partial L}{\partial Y}$).

- **A. Gradient w.r.t Filter Weights ($\frac{\partial L}{\partial W}$):** We need to find how much each weight contributed to the error. This is calculated as the convolution of the **Input** (X) and the **Error Gradient** (δ^{l+1}).

$$\frac{\partial L}{\partial W} = X * \delta^{l+1}$$

In matrix terms: This is the cross-correlation between the input feature map and the error map.

- **B. Gradient w.r.t Input ($\frac{\partial L}{\partial X}$ - passing error to previous layer):** To propagate error back to layer l , we calculate the gradient with respect to the input. This is equivalent to the "full" convolution of the **Error Gradient** (δ^{l+1}) and the **Rotated Filter** (W_{rot180}).

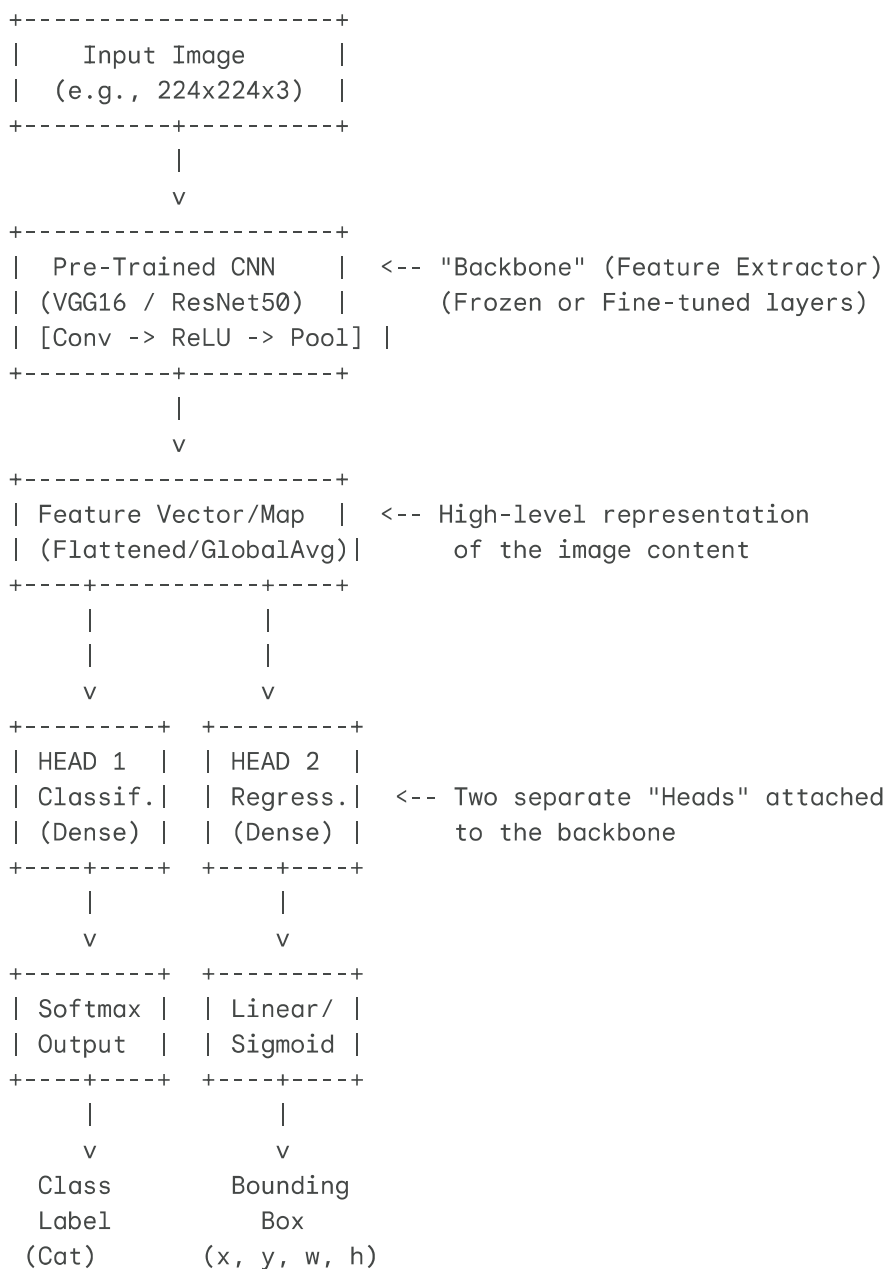
$$\delta^l = \delta^{l+1} * W_{rot180}$$

Reason for Rotation: To satisfy the chain rule mathematically during the backward pass of a convolution, the filter must be flipped 180 degrees horizontally and vertically.

7. How does a pre-trained Convolutional Neural Network handle Object Localization? Elaborate the steps with a diagram.

Object Localization involves identifying **what** is in an image (Classification) and **where** it is (Regression of bounding box coordinates). Pre-trained networks (like VGG16, ResNet50) trained on ImageNet are often used as "backbones" because they have already learned to extract powerful features.

Architecture Diagram



Detailed Steps:

Step 1: Feature Extraction (The Backbone)

- The input image is passed through a pre-trained CNN (e.g., VGG16 without its original top dense layers).
- This network acts as a **Feature Extractor**. It converts the raw pixels into a rich, high-dimensional feature vector that represents shapes, textures, and object parts.

Step 2: Attaching the Heads Instead of the original single output layer of the pre-trained network, we attach two parallel fully connected (dense) layers:

- **Classification Head:** A standard dense layer with a **Softmax** activation. It outputs a probability distribution over the classes (e.g., Dog, Cat, Car).
- **Regression (Localization) Head:** A dense layer with 4 output neurons (usually with **Linear** or **Sigmoid** activation). It outputs the bounding box coordinates: (x, y, w, h) or $(x_{min}, y_{min}, x_{max}, y_{max})$.

Step 3: Defining the Ground Truth For training, we need labels for both tasks:

- Class Label: y_{true} (e.g., "Cat").
- Bounding Box: $b_{true} = (x, y, w, h)$.

Step 4: Multi-Task Loss Calculation The network predicts both class probabilities (p) and box coordinates (b). We calculate a combined loss:

$$L_{total} = L_{class}(p, y_{true}) + \lambda \cdot L_{reg}(b, b_{true})$$

- L_{class} : Usually Categorical Cross-Entropy.
- L_{reg} : Usually Mean Squared Error (MSE), L1 Loss, or Smooth L1 Loss (which is less sensitive to outliers).
- λ (**Lambda**): A hyperparameter to balance the two losses (e.g., ensuring box accuracy doesn't dominate class accuracy).

Step 5: Training

- We use backpropagation to minimize L_{total} .
- Gradients flow back from both heads into the shared backbone, updating the weights to improve both classification and localization simultaneously.

8. Compute the gradient of the loss with respect to W_{hy} and W_{hh} using BPTT.

Given:

- **Hidden State Update:** $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$
- **Output (Implicit):** $\hat{y}_t = W_{hy}h_t$ (Assuming linear output for the given MSE loss structure).
- **Loss Function:** $L = \sum_{t=1}^T L_t = \sum_{t=1}^T \frac{1}{2}(y_t - \hat{y}_t)^2$

Part A: Gradient w.r.t Output Weights (W_{hy})

The weights W_{hy} affect the loss only at the current time step t . Therefore, there is no need to propagate through time for this matrix.

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^T \frac{\partial L_t}{\partial W_{hy}}$$

Using the Chain Rule:

$$\frac{\partial L_t}{\partial W_{hy}} = \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial W_{hy}}$$

1. **Error Term:** $\frac{\partial L_t}{\partial \hat{y}_t} = -(y_t - \hat{y}_t)$
2. **Output Derivative:** $\frac{\partial \hat{y}_t}{\partial W_{hy}} = h_t$

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^T -(y_t - \hat{y}_t) \cdot h_t$$

Part B: Gradient w.r.t Hidden Weights (W_{hh}) - Using BPTT

The weights W_{hh} are shared across all time steps. A change in W_{hh} affects h_1 , which affects h_2 , and so on. We must sum the gradients from all time steps.

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L_t}{\partial W_{hh}}$$

For a specific time step t , the loss L_t depends on h_t , which depends on h_{t-1} (which also used W_{hh}), and so on.

$$\frac{\partial L_t}{\partial W_{hh}} = \sum_{k=1}^t \frac{\partial L_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_k} \cdot \frac{\partial h_k}{\partial W_{hh}}$$

Step-by-Step Derivation:

1. Loss Gradient w.r.t h_t :

$$\frac{\partial L_t}{\partial h_t} = \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial h_t} = -(y_t - \hat{y}_t) \cdot W_{hy}$$

2. Immediate Gradient w.r.t W_{hh} at step k :

$$\frac{\partial h_k}{\partial W_{hh}} = (1 - h_k^2) \cdot h_{k-1}$$

(Note: derivative of $\tanh$ is $1 - \tanh^2$)

3. Backpropagation Term ($\frac{\partial h_t}{\partial h_k}$): This term represents the chain of dependencies from t back to k .

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=k+1}^t W_{hh} \cdot \text{diag}(1 - h_j^2)$$

Final Formula:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \sum_{k=1}^t [-(y_t - \hat{y}_t) W_{hy}] \left[\prod_{j=k+1}^t W_{hh} (1 - h_j^2) \right] [(1 - h_k^2) h_{k-1}]$$

9. Provide a comprehensive technical overview of a Convolutional Neural Network architecture.

A Convolutional Neural Network (CNN) is a deep learning architecture designed specifically for processing structured grid data (like images). It automatically learns spatial hierarchies of features.

1. Input Layer

- Holds the raw pixel values of the image.
- **Dimensions:** $Height \times Width \times Channels$ (e.g., $224 \times 224 \times 3$ for RGB).
- No learning happens here; it is just the data entry point.

2. Convolutional Layer (The Core)

- **Function:** Performs feature extraction.

- **Mechanism:** Slides small filters (kernels) across the input.
- **Parameters:**
 - **Filters/Kernels:** Learnable weights (e.g., 3×3 matrix) that detect patterns like edges, curves, or textures.
 - **Stride:** Step size of the filter.
 - **Padding:** Adding zeros around the border to preserve dimensions.
- **Output:** Generates "Feature Maps" (activation maps). If you have 32 filters, you get a stack of 32 feature maps.

3. Activation Layer (ReLU)

- **Function:** Introduces non-linearity.
- **Mechanism:** Usually applied immediately after convolution.
- **Operation:** $f(x) = \max(0, x)$. It replaces all negative pixel values in the feature map with zero.
- **Purpose:** Allows the network to learn complex, non-linear patterns (since convolution is a linear operation).

4. Pooling Layer (Downsampling)

- **Function:** Reduces spatial dimensions (Width/Height) while retaining important information.
- **Types:**
 - **Max Pooling:** Takes the largest value in a window (most common).
 - **Average Pooling:** Takes the average value.
- **Purpose:**
 - Reduces the number of parameters and computation.
 - Makes the network invariant to small translations/distortions in the image.

5. Fully Connected (Dense) Layer

- **Function:** High-level reasoning and classification.
- **Mechanism:** Neurons in this layer are connected to every activation in the previous layer.
- **Process:** The 3D feature maps from the final pooling layer are **Flattened** into a 1D vector before entering the FC layers.
- **Role:** It combines the local features extracted by the convolutional layers (e.g., "eye", "ear", "nose") to identify the global object (e.g., "Dog").

6. Output Layer

- **Function:** Produces the final prediction.
- **Mechanism:** Usually a Fully Connected layer with a specific activation function based on the task.
 - **Softmax:** For Multi-class classification (probabilities sum to 1).
 - **Sigmoid:** For Binary classification (probability between 0 and 1).
 - **Linear:** For Regression tasks (bounding box coordinates).

10. Describe the architecture of an Echo State Network (ESN) for smart grid energy demand forecasting.

In this scenario, an Echo State Network (ESN), a type of Recurrent Neural Network (Reservoir Computing), is used to predict energy demand based on real-time IoT data. The key characteristic of an ESN is that the hidden layer weights are sparse, random, and **fixed** (untrained).

Architecture Overview:

1. Input Layer:

- **Role:** Receives the raw feature vectors from the IoT sensors.
- **Input Features:** [*Current_Energy, Weather_Conditions, Time_of_Day*].
- **Function:** Projects these low-dimensional features into the high-dimensional reservoir using fixed, random weights (W_{in}).

2. Reservoir (The "Echo" Layer):

- **Role:** Acts as a non-linear, high-dimensional dynamical memory. It transforms the inputs into a rich "reservoir state" that retains a history (echo) of previous inputs.
- **Structure:** A large collection of sparsely connected neurons (hidden units) with recurrent connections.
- **Weights (W_{res}):** Randomly initialized and **remain fixed** throughout the entire process. They are scaled so the "spectral radius" is < 1 to ensure the "echo state property" (stability).

3. Output Layer (Readout):

- **Role:** Maps the complex state of the reservoir to the specific target variable (Energy Demand for next hour).
- **Training:** This is the **only trainable part** of the network. A simple linear regression (usually Ridge Regression) is used to find the optimal weights (W_{out}) that map the reservoir state to the output.

- **Advantage:** Because only the output layer is trained, training is extremely fast and computationally efficient compared to standard RNNs or LSTMs (no Backpropagation Through Time required).

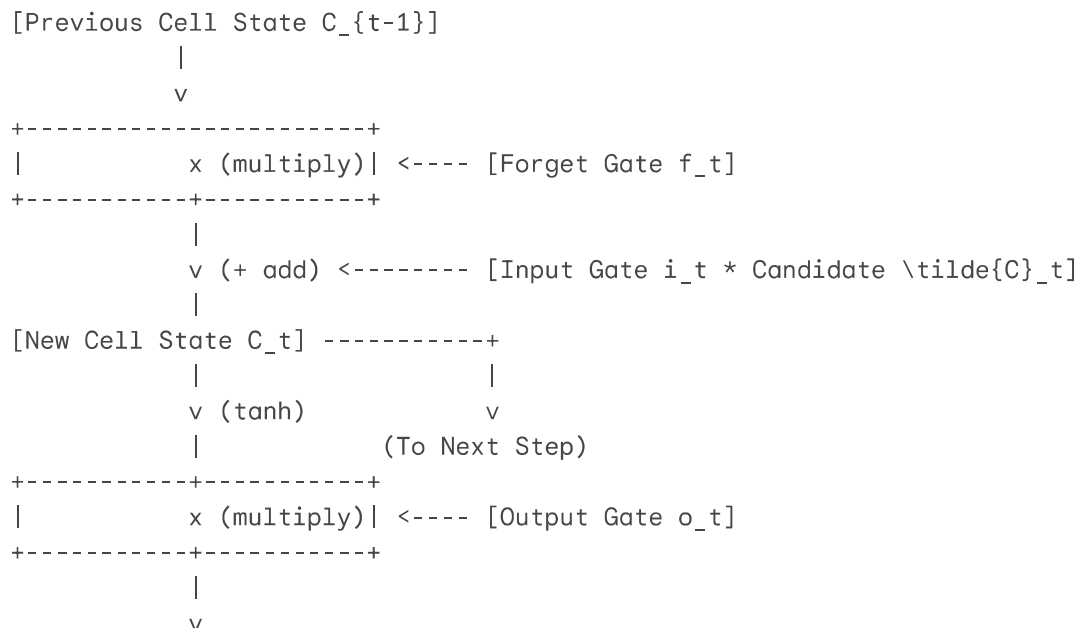
11. Discuss how an LSTM can be used for grammatical error detection in the sentence "Captain Rohit fail in BGT Test Series" and provide the LSTM cell architecture and equations.

Application of LSTM:

The sentence "Captain Rohit fail in BGT Test Series" contains a subject-verb agreement error. "Captain Rohit" is a third-person singular subject, so the verb should be "fails" or "failed".

- **Language Modeling:** An LSTM trained on correct English text acts as a Language Model. It calculates the probability of the next word given the previous context.
- **Detection Mechanism:**
 1. The LSTM reads "Captain".
 2. It updates its state and predicts the next likely words.
 3. It reads "Rohit". The internal state now encodes a singular proper noun subject.
 4. It predicts the next word. The probability of "failed" or "fails" would be high. The probability of the base form "fail" would be very low.
 5. **Correction:** Comparing the low probability of the actual word ("fail") against high-probability alternatives allows the system to flag the error and suggest "failed/fails" as a correction.

LSTM Cell Architecture Diagram



[New Hidden State h_t]

Mathematical Equations for LSTM Cell State

1. **Forget Gate (f_t):** Decides what to throw away from the previous state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. **Input Gate (i_t):** Decides which values to update.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

3. **Candidate Cell State ($\tilde{C}_t$):** Creates a vector of new candidate values.

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

4. **Cell State Update (C_t):** The core equation updating the long-term memory.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

5. **Output Gate (o_t):** Decides what part of the cell state to output.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

6. **Hidden State (h_t):** The final output for this time step.

$$h_t = o_t * \tanh(C_t)$$

12. Provide a detailed explanation of the mathematical formulations for Bidirectional and Multilayer Recurrent Neural Networks.

A. Bidirectional RNNs (BRNNs)

Bidirectional RNNs address the limitation of standard RNNs which only look at past context. In tasks like speech recognition or translation, knowing what comes *after* a word is as important as knowing what came *before*.

Architecture: A BRNN connects two hidden layers of opposite directions to the same output.

1. **Forward Layer:** Processes sequence from $t = 1$ to T .
2. **Backward Layer:** Processes sequence from $t = T$ to 1.

Mathematical Formulations:

1. **Forward Pass (Left-to-Right):** Calculates the hidden state $\vec{h}_t$ based on the previous state $\vec{h}_{t-1}$ and current input x_t .

$$\vec{h}_t = \sigma(W_{x\vec{h}}x_t + W_{\vec{h}\vec{h}}\vec{h}_{t-1} + b_{\vec{h}})$$

2. **Backward Pass (Right-to-Left):** Calculates the hidden state $\overleftarrow{h}_t$ based on the *future* state $\overleftarrow{h}_{t+1}$ and current input x_t .

$$\overleftarrow{h}_t = \sigma(W_{x\overleftarrow{h}}x_t + W_{\overleftarrow{h}\overleftarrow{h}}\overleftarrow{h}_{t+1} + b_{\overleftarrow{h}})$$

3. **Output Equation:** The final output y_t at time t combines information from both directions.

$$y_t = W_{\vec{h}y}\vec{h}_t + W_{\overleftarrow{h}y}\overleftarrow{h}_t + b_y$$

B. Multilayer RNNs (Stacked RNNs)

Multilayer RNNs increase the capacity of the model to learn hierarchical features. Lower layers capture low-level features (e.g., syntax), while higher layers capture high-level abstract features (e.g., semantics).

Architecture: Multiple RNN layers are stacked on top of each other. The **output** (hidden state) of layer $l - 1$ becomes the **input** for layer l .

State Transition Equation: For a network with L layers, let $h_t^{(l)}$ be the hidden state of layer l at time t .

$$h_t^{(l)} = \sigma(W_{h^{(l-1)}h^{(l)}}h_t^{(l-1)} + W_{h^{(l)}h^{(l)}}h_{t-1}^{(l)} + b^{(l)})$$

- $h_t^{(l-1)}$: The input from the layer below (at the same time step).
- $h_{t-1}^{(l)}$: The recurrent input from the same layer (from the previous time step).
- *Note:* For the first layer ($l = 1$), the input is the actual data x_t .

13. Consider a machine translation task (English to Kannada). Identify the most appropriate type of RNN and explain its architecture.

Identified Model: Sequence-to-Sequence (Seq2Seq) Model with Attention (Encoder-Decoder).

Justification:

1. **Variable Lengths:** The input sentence (English) and output sentence (Kannada) often have different lengths (e.g., "How are you?" -> "Hegidira?" is 3 words to 1 word). Standard RNNs require one-to-one mapping, which fails here.
2. **Semantic Context:** Translation requires understanding the entire sentence context before generating the first word of the target language. The Encoder-Decoder architecture captures the full semantic meaning in a context vector.

Architecture Explanation:

1. **The Encoder:**
 - **Type:** Often a Bidirectional LSTM or GRU.
 - **Function:** Reads the input English sentence one token at a time.
 - **Output:** It produces a final **Context Vector** (often the final hidden state) that encapsulates the summary of the entire English sentence.
2. **The Decoder:**
 - **Type:** Unidirectional LSTM/GRU.
 - **Function:** Takes the context vector as its initial state. It generates the Kannada translation one word at a time.
 - **Process:** At each step, it uses its previous hidden state and the previously generated word to predict the next word.
3. **Attention Mechanism (Crucial Add-on):**
 - Instead of relying on a single static context vector (which acts as a bottleneck for long sentences), Attention allows the Decoder to "look back" at specific parts of the English sentence that are relevant to the Kannada word currently being generated.

14. Convolution Operation

Given:

- **Filter (3×3):**

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

(This is a Vertical Edge Detector)

- **Input (6×6):**

$$I = \begin{bmatrix} 3 & 0 & 1 & 2 & 7 & 4 \\ 1 & 5 & 8 & 9 & 3 & 1 \\ 2 & 7 & 2 & 5 & 1 & 3 \\ 0 & 1 & 3 & 1 & 7 & 8 \\ 4 & 2 & 1 & 6 & 2 & 8 \\ 2 & 4 & 5 & 2 & 3 & 9 \end{bmatrix}$$

- **Parameters:** Stride = 1, Zero Padding (Typically means Padding $P = 1$ to maintain size, but we calculate the **Valid Output** for core feature analysis, corresponding to $P = 0$).

i) **Perform the convolution operation (Valid Output):** Output Dimensions =

$$\left\lfloor \frac{6-3}{1} \right\rfloor + 1 = 4 \times 4.$$

Calculations (Sample):

- **Top-Left (0,0):**

$$1(3) + 0(0) - 1(1) + 1(1) + 0(5) - 1(8) + 1(2) + 0(7) - 1(2) = 2 - 7 + 0 = -5$$

- **Top-Right (0,3):**

$$1(2) + 0(7) - 1(4) + 1(9) + 0(3) - 1(1) + 1(5) + 0(1) - 1(3) = -2 + 8 + 2 = 8$$

- **Bottom-Right (3,3):**

$$1(1) + 0(7) - 1(8) + 1(6) + 0(2) - 1(8) + 1(2) + 0(3) - 1(9) = -7 - 2 - 7 = -16$$

Final Output Matrix:

$$\text{Output} = \begin{bmatrix} -5 & -4 & 0 & 8 \\ -10 & -2 & 2 & 3 \\ 0 & -2 & -4 & -7 \\ -3 & -2 & -3 & -16 \end{bmatrix}$$

Discussion: The output highlights **vertical edges** in the input image.